

2dCFT Numerics: Code Structure and Conventions

May 14, 2026

1 Purpose

The purpose of the code structure document is to provide a clear and organized overview of the codebase. We will check the Pentagon equation as well as the idempotency relation for the Fusion kernels. We will first review the spacelike results at rational central charge. Then, we will move on to the timelike results and check the Pentagon. It will amount to use the Virasoro Wick Rotation. We first recall the form of the Pentagon equation and the idempotency relation. The pentagon equation reads

$$\int_{i\mathbb{R}} \frac{dP_s}{i} \mathbb{F}_{P_q, P_4}^{(b)} \begin{bmatrix} P_s & P_3 \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_o, P_s}^{(b)} \begin{bmatrix} P_1 & P_2 \\ P_u & P_q \end{bmatrix} \mathbb{F}_{P_s, P_t}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} = \mathbb{F}_{P_o, P_4}^{(b)} \begin{bmatrix} P_1 & P_t \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_q, P_t}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_o & P_5 \end{bmatrix} \quad (1)$$

Ioannis advised me to switch variables $P_s = ip_s$ so that making this change of variable the Pentagon equation reads

$$\begin{aligned} & \int_{\mathbb{R}} dp_s \mathbb{F}_{P_q, P_4}^{(b)} \begin{bmatrix} ip_s & P_3 \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_o, ip_s}^{(b)} \begin{bmatrix} P_1 & P_2 \\ P_u & P_q \end{bmatrix} \mathbb{F}_{ip_s, P_t}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \\ & = \mathbb{F}_{P_o, P_4}^{(b)} \begin{bmatrix} P_1 & P_t \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_q, P_t}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_o & P_5 \end{bmatrix} \end{aligned} \quad (2)$$

For spacelike Liouville, the fusion kernels on the real line $p \in \mathbb{R}$, *i.e.* $P \in i\mathbb{R}$ are real. In this expression, we have unknowns $P_o, P_q, P_1, P_2, P_3, P_4, P_5, P_t$ as well as b parametrized as $b = \sqrt{\frac{m}{n}}$ (we also define $s = \sqrt{mn}$). We integrate on p_s . We will check it for imaginary values of the P 's. The idempotency relation reads

$$\int_{i\mathbb{R}} \frac{dP}{i} \mathbb{F}_{P_s, P}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \mathbb{F}_{P, P_s'}^{(b)} \begin{bmatrix} P_3 & P_4 \\ P_2 & P_1 \end{bmatrix} = \frac{1}{2} (\delta(P_s - P_s') + \delta(P_s + P_s')) . \quad (3)$$

2 Expressions

2.1 Special functions

We first need the expression of the special functions $G_{m,n}$ and $\tilde{G}_{m,n}$. The easiest one is $G_{m,n}$ which is given by the following product

$$G_{m,n}(x) = \prod_{k=0}^{m-1} \prod_{l=0}^{n-1} G\left(x + \frac{k}{m} + \frac{l}{n}\right) , \quad (4)$$

where G is the Barnes G-function. It should satisfy (consistency check)

$$\frac{G_{m,n}(x + 1/n)}{G_{m,n}(x)} = m^{1/2 - mx} (2\pi)^{(m-1)/2} \Gamma(mx) . \quad (5)$$

The expression of $\tilde{G}_{m,n}$. We first define $\tilde{G}$ as

$$\tilde{G}(x) = \frac{G(1+x)}{G(1-x)}, \quad (6)$$

and then

$$\begin{aligned} \tilde{G}_{m,n}(x) &= \prod_{k=0}^{m-1} \prod_{l=0}^{n-1} \tilde{G}\left(x - x_{(k,l)}^{m,n}\right), \\ x_{(k,l)}^{m,n} &= 1 - \left(\frac{k+1/2}{m} + \frac{l+1/2}{n}\right). \end{aligned} \quad (7)$$

Finally, we can express the double gamma function

$$\Gamma_b(x) = \gamma_{m,n} \frac{(mn)^{\frac{x}{4}(Q-x)} (2\pi)^{\frac{x\sqrt{mn}}{2} - \frac{m+n}{4}}}{G_{m,n}\left(\frac{x}{\sqrt{mn}}\right)}, \quad (8)$$

with $\gamma_{m,n} = (mn)^{-\frac{Q^2}{16}} G_{m,n}\left(\frac{1}{2m} + \frac{1}{2n}\right)$. There are few properties of these special functions that we can check numerically (consistency checks):

- Complex conjugation:

$$\overline{\tilde{G}_{m,n}(x)} = \tilde{G}_{m,n}(\bar{x}). \quad (9)$$

- Inversion relations:

$$\tilde{G}_{m,n}(x)^{-1} = \tilde{G}_{m,n}(-x). \quad (10)$$

- Identity limit:

$$\Gamma_b\left(\frac{Q}{2}\right) = 1. \quad (11)$$

- Shift relations:

$$\frac{\Gamma_b(x + b^{\pm 1})}{\Gamma_b(x)} = \frac{\sqrt{2\pi} b^{\pm b^{\pm 1} x \mp \frac{1}{2}}}{\Gamma(b^{\pm 1} x)}. \quad (12)$$

2.2 CFT data

Spacelike case We choose the following normalization scheme for the 2 point and the 3 point structure constants.

$$\begin{aligned} \langle V_{P_1}(0) V_{P_2}(1) \rangle &= \frac{1}{\rho_0^{(b)}(P_1)} (\delta(P_1 - P_2) + \delta(P_1 + P_2)), \\ \langle V_{P_1}(0) V_{P_2}(1) V_{P_3}(\infty) \rangle &= C_0^{(b)}(P_1, P_2, P_3), \end{aligned} \quad (13)$$

where

$$\begin{aligned} \rho_0^{(b)}(P) &= -4\sqrt{2} \sin(2\pi b P) \sin(2\pi P/b), \\ C_0^{(b)}(P_1, P_2, P_3) &= \frac{\Gamma_b(2Q) \Gamma_b\left(\frac{Q}{2} \pm P_1 \pm P_2 \pm P_3\right)}{\sqrt{2} \Gamma_b(Q)^3 \prod_{j=1}^3 \Gamma_b(Q \pm 2P_j)}. \end{aligned} \quad (14)$$

There is an important consistency check. The identity limit of the 3 point structure constant in spacelike Liouville should be consistent with the 2 point structure constant. We have

$$\lim_{P_3 \rightarrow Q/2} C_0^{(b)}(P_1, P_2, P_3) = \frac{1}{\rho_0^{(b)}(P_1)} (\delta(P_1 - P_2) + \delta(P_1 + P_2)). \quad (15)$$

Timelike case We choose a normalization for which timelike Liouville CFT data is barely the inverse of the spacelike one. We have

$$\begin{aligned}\hat{B}(P_1) &:= \frac{1}{\hat{\rho}_0(P_1)} = \frac{2\rho_0(iP_1)}{P_1^2}, \\ \hat{C}_0(P_1, P_2, P_3) &= \frac{1}{C_0^{(b)}(iP_1, iP_2, iP_3)}.\end{aligned}\tag{16}$$

2.3 Spacelike expressions

Fusion kernel We start by the different parameters defining the quantum modular fusion polynomial of the fusion kernel. We first define $c_{mn}(x) = \cos(2\pi s x)$ and $s_{mn}(x) = \sin(2\pi s x)$ as well as

$$a_i = P_i - \frac{m+n}{2s}.\tag{17}$$

The quantum modular fusion determinant is easily expressed in terms of the determinant of the following Gram matrix. We define the symmetric gram matrix

$$\mathcal{G}_{m,n}^{(f)} = \begin{bmatrix} 1 & -c_{mn}(a_2) & -c_{mn}(a_3) & c_{mn}(a_s) \\ -c_{mn}(a_2) & 1 & c_{mn}(a_t) & -c_{mn}(a_1) \\ -c_{mn}(a_3) & c_{mn}(a_t) & 1 & -c_{mn}(a_4) \\ c_{mn}(a_s) & -c_{mn}(a_1) & -c_{mn}(a_4) & 1 \end{bmatrix}\tag{18}$$

The quantum modular fusion determinant takes the simple form

$$D^{(f)} = \frac{-4 \det \mathcal{G}_{m,n}^{(f)}}{s_{mn}(P_s)^2 s_{mn}(P_t)^2}.\tag{19}$$

This would be interesting to display in the 2D plot the lines where the determinant $D^{(f)}$ is -1 with external momenta fixed and P_s, P_t varying. Finally, we also need the expression of the quantum modular fusion polynomial to get the roots $z_{\pm}^{(f)}$. We have

$$P_{m,n}^{(f)}(z, \vec{P}) = \alpha_{m,n}^{(f)} z^2 + \beta_{m,n}^{(f)} z + \gamma_{m,n}^{(f)},\tag{20}$$

with

$$\begin{aligned}\alpha_{m,n}^{(f)} = & + \left[\exp \left(2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (P_s + P_t - P_1 - P_2 - P_3 - P_4) \right) \right) \right. \\ & + \exp \left(2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (P_s + P_t + P_1 + P_2 + P_3 + P_4) \right) \right) \\ & + \exp \left(2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (-P_s - P_t - P_1 + P_2 - P_3 + P_4) \right) \right) \\ & \left. + \exp \left(2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (-P_s - P_t + P_1 - P_2 + P_3 - P_4) \right) \right) \right] \\ & - \left[\exp \left(2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (-P_s + P_t - P_1 - P_2 + P_3 + P_4) \right) \right) \right. \\ & + \exp \left(2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (-P_s + P_t + P_1 + P_2 - P_3 - P_4) \right) \right) \\ & + \exp \left(2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (P_s - P_t - P_1 + P_2 + P_3 - P_4) \right) \right) \\ & \left. + \exp \left(2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s} (P_s - P_t + P_1 - P_2 - P_3 + P_4) \right) \right) \right],\end{aligned}\tag{21}$$

and

$$\beta_{m,n}^{(f)} = 4[s_{mn}(P_1)s_{mn}(P_3) + s_{mn}(P_2)s_{mn}(P_4) + s_{mn}(P_s)s_{mn}(P_t)] . \quad (22)$$

The expression of $\gamma_{m,n}^{(f)}$ is the complex conjugate of $\alpha_{m,n}^{(f)}$ when the momenta are real.

$$\begin{aligned} \gamma_{m,n}^{(f)} = & + \left[\exp \left(2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(P_s + P_t - P_1 - P_2 - P_3 - P_4) \right) \right) \right. \\ & + \exp \left(-2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(P_s + P_t + P_1 + P_2 + P_3 + P_4) \right) \right) \\ & + \exp \left(-2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(-P_s - P_t - P_1 + P_2 - P_3 + P_4) \right) \right) \\ & + \exp \left(-2\pi i s^2 \left(\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(-P_s - P_t + P_1 - P_2 + P_3 - P_4) \right) \right) \Big] \\ & - \left[\exp \left(-2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(-P_s + P_t - P_1 - P_2 + P_3 + P_4) \right) \right) \right. \\ & + \exp \left(-2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(-P_s + P_t + P_1 + P_2 - P_3 - P_4) \right) \right) \\ & + \exp \left(-2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(P_s - P_t - P_1 + P_2 + P_3 - P_4) \right) \right) \\ & + \exp \left(-2\pi i s^2 \left(-\frac{m^{-1} + n^{-1}}{4} + \frac{1}{2s}(P_s - P_t + P_1 - P_2 - P_3 + P_4) \right) \right) \Big] , \end{aligned} \quad (23)$$

We will not use it to compute the roots of the polynomials but it might serve as a consistency check. Indeed, we have

$$z_{\pm}^{(f)} = \frac{-\beta_{m,n}^{(f)} \pm 2is_{mn}(P_t)s_{mn}(P_s)\sqrt{D(f)}}{2\alpha_{m,n}^{(f)}} = \frac{-\beta_{m,n}^{(f)} \pm \sqrt{\Delta(f)}}{2\alpha_{m,n}^{(f)}} . \quad (24)$$

We barely have everything.

We define the $\pm$ kernels whose expressions for rational charge are given in your paper Ioannis <https://arxiv.org/pdf/2512.03172>.

$$\mathbb{F}_{P_s, P_t}^{(\pm)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} = \pm \frac{F_b(P_1, P_2, P_3, P_4 | P_s, P_t)}{2i \sin(2\pi s P_t) \sin(2\pi s P_s)} \frac{1}{\sqrt{D(f)}} \sum_{k=0}^{s^2-1} \mathcal{F}_b \left(\frac{\log z_{\pm}^{(f)}}{2\pi i s^2} + \frac{k}{s^2} \right) . \quad (25)$$

We have

$$\mathcal{F}_b(u) = \frac{\text{Numerator}(\mathcal{F}_b)}{\text{Denominator}(\mathcal{F}_b)} , \quad (26)$$

with

$$\begin{aligned} \text{Numerator}(\mathcal{F}_b) = & \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s}(-P_s + P_t - P_1 - P_2 + P_3 + P_4) \right) \\ & \times \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s}(-P_s + P_t + P_1 + P_2 - P_3 - P_4) \right) \\ & \times \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s}(P_s - P_t - P_1 + P_2 + P_3 - P_4) \right) \\ & \times \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s}(P_s - P_t + P_1 - P_2 - P_3 + P_4) \right) , \end{aligned} \quad (27)$$

and

$$\begin{aligned}
\text{Denominator}(\mathcal{F}_b) = & \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s} (P_s + P_t - P_1 - P_2 - P_3 - P_4) \right) \\
& \times \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s} (P_s + P_t + P_1 + P_2 + P_3 + P_4) \right) \\
& \times \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s} (-P_s - P_t - P_1 + P_2 - P_3 + P_4) \right) \\
& \times \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2s} (-P_s - P_t + P_1 - P_2 + P_3 - P_4) \right).
\end{aligned} \tag{28}$$

Next, we also have

$$F_b(u) = \frac{\text{Numerator}(\mathcal{F}_b)}{\text{Denominator}(\mathcal{F}_b)}, \tag{29}$$

with

$$\begin{aligned}
\text{Numerator}(\mathcal{F}_b) = & i(2\pi)^{4s^2-(m+n)} G_{m,n} \left(\frac{2P_t}{s} \right) G_{m,n} \left(\frac{-2P_t}{s} \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_1 - P_2 + P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_1 + P_2 + P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_1 + P_2 - P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_1 - P_2 - P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_3 - P_4 + P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_3 + P_4 + P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_3 + P_4 - P_s) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_3 - P_4 - P_s) \right),
\end{aligned} \tag{30}$$

and

$$\begin{aligned}
\text{Denominator}(\mathbb{F}_b) = & s G_{m,n} \left(m^{-1} + n^{-1} + \frac{2P_s}{s} \right) G_{m,n} \left(m^{-1} + n^{-1} - \frac{2P_s}{s} \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_2 + P_3 + P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_2 - P_3 - P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_2 - P_3 + P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_2 + P_3 - P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_1 + P_4 + P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (P_1 - P_4 - P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_1 - P_4 + P_t) \right) \\
& \times G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} + \frac{1}{s} (-P_1 + P_4 - P_t) \right) .
\end{aligned} \tag{31}$$

Consistency chekcs for the fusion kernel One non trivial check that we can try to verify is the following symmetry of $\mathcal{F}_b(u)$ namely that is invariant under $P_s \rightarrow P_3, P_3 \rightarrow P_s, P_t \rightarrow P_1, P_1 \rightarrow P_t$. A second non trivial is the fact that the sum defined in the $\pm$ kernels is single-valued and thus does not depend on the branch of the logarithm i.e

$$\sum_{k=0}^{s^2-1} \mathcal{F}_b \left(\frac{\log z_{\pm}^{(f)}}{2\pi i s^2} + \frac{k}{s^2} \right) = \sum_{k=0}^{s^2-1} \mathcal{F}_b \left(\frac{\log z_{\pm}^{(f)}}{2\pi i s^2} + \frac{k+\ell}{s^2} \right) , \tag{32}$$

for all ℓ in $\mathbb{Z}$.

A simple but non trivial property of the kernels is their transformation rules under reflection of internal momenta. Basically, $\mathbb{F}^{(\pm)}$ should get exchanged under reflection of the internal momenta eg $P_s \rightarrow -P_s$ in the spacelike case.

A highly non trivial property is the mixing property which we write

$$\mathbb{F}_{P_s, P_t}^{(\eta)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} = \frac{\mathbb{F}_{\mathbb{I}, P_t}^{(TV)} \begin{bmatrix} P_2 & P_3 \\ P_2 & P_3 \end{bmatrix}}{\mathbb{F}_{\mathbb{I}, P_1}^{(TV)} \begin{bmatrix} P_2 & P_s \\ P_2 & P_s \end{bmatrix}} \mathbb{F}_{P_3, P_1}^{(\eta)} \begin{bmatrix} P_2 & P_s \\ P_t & P_4 \end{bmatrix} , \tag{33}$$

where TV stands for the Teschner/Vartanov kernel. Before checking the pentagon equation for the spacelike kernels, we will first verify a relation that was written by Ioannis and Sylvain in the paper concerning the $c = 25$ fusion kernels. Precisely, the relation reads

$$\mathbb{F}_{P_s, P_t}^{(\pm)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} = \pm i \frac{P_t}{P_s} 16^{P_t^2 - P_s^2} e^{\pm 2\pi i P_s P_t} . \tag{34}$$

After checking that we are ready to implement the pentagon identity for the spacelike kernels. In order to test if it satisfied also for $F^{(\pm)}$ and only for the Teschner-Vartanov Kernel, we will

implement the following:

$$\begin{aligned} & \int_{\mathbb{R}} dp_s \mathbb{F}_{P_q, P_4} \begin{bmatrix} ip_s & P_3 \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_o, ip_s} \begin{bmatrix} P_1 & P_2 \\ P_u & P_q \end{bmatrix} \mathbb{F}_{ip_s, P_t}^{(TV)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \\ &= \mathbb{F}_{P_o, P_4} \begin{bmatrix} P_1 & P_t \\ P_u & P_5 \end{bmatrix} \mathbb{F}_{P_q, P_t} \begin{bmatrix} P_2 & P_3 \\ P_o & P_5 \end{bmatrix} \end{aligned} \quad (35)$$

where $\mathbb{F}$ denotes a generic kernel (it could be $\mathbb{F}^{(\pm)}$ or the Teschner-Vartanov one $\mathbb{F}^{(TV)}$ or something else that has the same arguments).

Studying a particular case of the Pentagon identity analytically Let's first discuss the case where all the free momenta are equal to $1/4$ and we take the kernels to be the Teschner-Vartanov one. The RHS reads

$$\left(\mathbb{F}_{1/4, 1/4}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} \right)^2 = \sin^2(\pi/8) = \frac{2 - \sqrt{2}}{4}. \quad (36)$$

The LHS reads

$$\int_{i\mathbb{R}} \frac{dP_s}{i} \mathbb{F}_{1/4, 1/4}^{(TV)} \begin{bmatrix} P_s & 1/4 \\ 1/4 & 1/4 \end{bmatrix} \mathbb{F}_{1/4, P_s}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} \mathbb{F}_{P_s, 1/4}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix}. \quad (37)$$

We would like to show something like

$$\int \frac{dP_s}{i} \mathcal{I}(P_s) = 2\pi \sum_{n \geq 0} c_n = \sin^2\left(\frac{\pi}{8}\right), \quad (38)$$

where c_n are the residues of $\mathcal{I}(P_s)$ the integrand in the LHS. The two last terms in the integrand can be simplified using the relation (34)

$$\mathbb{F}_{1/4, P_s}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} \mathbb{F}_{P_s, 1/4}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} = \sin^2\left(\frac{\pi P_s}{2}\right). \quad (39)$$

We thus observe from (36) and (39) that it suffices that the first term in the integrand imposes a kind of delta function at $P_s = 1/4$ for the pentagon identity to be satisfied in this specific case. Let's look at the expression of the first term. We use the mixing property (33) as well as the symmetry under exchange of external momenta

$$\begin{aligned} \mathbb{F}_{1/4, 1/4}^{(TV)} \begin{bmatrix} P_s & 1/4 \\ 1/4 & 1/4 \end{bmatrix} &= \mathbb{F}_{1/4, 1/4}^{(TV)} \begin{bmatrix} 1/4 & P_s \\ 1/4 & 1/4 \end{bmatrix} = \mathbb{F}_{P_s, 1/4}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix} \frac{\mathbb{F}_{\mathbb{I}, 1/4}^{(TV)} \begin{bmatrix} 1/4 & P_s \\ 1/4 & P_s \end{bmatrix}}{\mathbb{F}_{\mathbb{I}, 1/4}^{(TV)} \begin{bmatrix} 1/4 & 1/4 \\ 1/4 & 1/4 \end{bmatrix}} \\ &= -\frac{1}{4P_s} 16^{\left(\frac{1}{4}\right)^2 - P_s^2} \sin\left(\frac{\pi P_s}{2}\right) \frac{\mathbb{F}_{\mathbb{I}, 1/4}^{(TV)} \begin{bmatrix} 1/4 & P_s \\ 1/4 & P_s \end{bmatrix}}{-\frac{1}{2Q} 16^{\left(\frac{1}{4}\right)^2 - \frac{Q^2}{4}} \sin\left(\frac{\pi Q}{4}\right)} \\ &= \frac{Q}{2P_s} 16^{\frac{Q^2}{4} - P_s^2} \frac{\sin\left(\frac{\pi P_s}{2}\right)}{\sin\left(\frac{\pi Q}{4}\right)} \mathbb{F}_{\mathbb{I}, 1/4}^{(TV)} \begin{bmatrix} 1/4 & P_s \\ 1/4 & P_s \end{bmatrix}. \end{aligned} \quad (40)$$

Finally, we can use the expression of the Teschner-Vartanov kernel for an identity internal momentum in terms of bulk CFT data

$$\mathbb{F}_{\mathbb{I}, P_1}^{(TV)} \begin{bmatrix} P_2 & P_3 \\ P_2 & P_3 \end{bmatrix} = \frac{1}{2} \rho_0(P_1) C_0(P_1, P_2, P_3), \quad (41)$$

yielding for our specific case

$$\mathbb{F}_{\mathbb{I},1/4}^{(TV)} \begin{bmatrix} 1/4 & P_s \\ 1/4 & P_s \end{bmatrix} = \frac{1}{2} \rho_0(1/4) C_0(1/4, 1/4, P_s) = -2\sqrt{2} \sin\left(\frac{\pi b}{2}\right) \sin\left(\frac{\pi}{2b}\right) C_0(1/4, 1/4, P_s). \quad (42)$$

All in all the integrand of the LHS is easily expressed in terms of the bulk structure constants

$$\mathcal{I}(P_s) = -\frac{Q\sqrt{2}}{P_s} 16^{\frac{Q^2}{4}-P_s^2} \frac{\sin^3\left(\frac{\pi P_s}{2}\right)}{\sin\left(\frac{\pi Q}{4}\right)} \sin\left(\frac{\pi b}{2}\right) \sin\left(\frac{\pi}{2b}\right) C_0(1/4, 1/4, P_s). \quad (43)$$

Importantly, this is supposed to hold for $c = 25 \Rightarrow b = 1, Q = 2$. We will focus in such a case in what follows. Thus we can further simplify to

$$\begin{aligned} \mathcal{I}(P_s) &= -\frac{2\sqrt{2}}{P_s} 16^{1-P_s^2} \sin^3\left(\frac{\pi P_s}{2}\right) C_0(1/4, 1/4, P_s) \\ &= -\frac{2\sqrt{2}}{P_s} 16^{1-P_s^2} \sin^3\left(\frac{\pi P_s}{2}\right) \\ &\quad \times \frac{\Gamma_1(4)}{\sqrt{2}\Gamma_1(2)^3} \frac{\Gamma_1\left(\frac{1}{2}-P_s\right) \Gamma_1\left(\frac{1}{2}+P_s\right) \Gamma_1\left(\frac{3}{2}-P_s\right) \Gamma_1\left(\frac{3}{2}+P_s\right) (\Gamma_1(1-P_s))^2 (\Gamma_1(1+P_s))^2}{\Gamma_1(2+2P_s) \Gamma_1(2-2P_s) (\Gamma_1(3/2))^2 (\Gamma_1(5/2))^2}. \end{aligned} \quad (44)$$

In the second equality, we substitute for the bulk structure constants expression for $Q = 2$. In order to perform the integral, this is useful to analyse the pole structure of the bulk structure constants. Recall now that the double Gamma function has no zeroes and simple poles at $x \in -\mathbb{N}b - \mathbb{N}b^{-1} = -\mathbb{N}$ (in the case $b = 1$). From (44), we thus conclude that the set of poles at the numerator of the bulk structure constants is

$$\mathcal{P} = \pm \frac{1}{4} \pm \frac{1}{4} \pm \mathbb{N}^*, \quad (45)$$

while the set of poles of the denominator is

$$\mathcal{Z} = \pm \left(1 + \frac{\mathbb{N}}{2}\right). \quad (46)$$

After a bit of algebra (and if you carefully analyse the multiplicity of the poles), one can check that the structure constants have simple poles at

$$P_s \in \frac{\mathbb{N}^*}{2} \sqcup \left(-\frac{\mathbb{N}^*}{2}\right). \quad (47)$$

This is not the end of the story since the prefactor has triple zeroes at $P_s \in 2\mathbb{N}$ except at $P_s = 0$ where it only has a double zero. All in all, the integrand has the following structure of poles and zeroes:

- Simple poles for P_s odd or half integer.
- Double zeroes for P_s even.

Let's analyse the residue of (44) at $P_s = 1/2$. The prefactor in front of $C_0(1/4, 1/4, P_s)$ at $P_s = 1/2$ is simply

$$-4\sqrt{2} \times 8 \times \left(\frac{\sqrt{2}}{2}\right)^3 = -16.$$

There is also a prefactor in $C_0(1/4, 1/4, P_s)$ which does not contribute to the pole ($b = 1$)

$$\frac{\Gamma_1(4)}{\sqrt{2}\Gamma_1(2)^3} \frac{\Gamma_1(1)^2 \Gamma_1(2) \Gamma_1(1/2)^2 \Gamma_1(3/2)^2}{\Gamma_1(3) \Gamma_1(1) \Gamma_1(3/2)^2 \Gamma_1(5/2)^2} = \frac{1}{64\sqrt{\pi}}. \quad (48)$$

Finally,

$$\operatorname{Res}_{P_s=1/2, b=1} \Gamma_1 \left(\frac{1}{2} - P_s \right) = -\frac{1}{\sqrt{2\pi}}. \quad (49)$$

Thus,

$$\operatorname{Res}_{P_s=1/2, b=1} \mathcal{I}(P_s) = \frac{1}{4\sqrt{2\pi}}. \quad (50)$$

Let's now analyse the residue at $P_s = 1$. The prefactor in front of $C_0(1/4, 1/4, P_s)$ simplifies to

$$-2\sqrt{2}$$

The factor inside $C_0(1/4, 1/4, P_s)$ which does not contribute to the pole is

$$\frac{\Gamma_1(4)}{\sqrt{2}\Gamma_1(2)^3} \frac{\Gamma_1(-1/2)\Gamma_1(3/2)\Gamma_1(5/2)\Gamma_1(1/2)\Gamma_1(2)^2}{\Gamma_1(4)\Gamma_1(3/2)^2\Gamma_1(5/2)^2} = -\frac{1}{8\sqrt{\pi}}. \quad (51)$$

Fianlly, we also have

$$\frac{\Gamma_1(1-P_s)^2}{\Gamma_1(2-2P_s)} \underset{P_s \rightarrow 1}{\sim} \sqrt{\frac{2}{\pi}} \frac{1}{1-P_s} \Rightarrow \operatorname{Res}_{P_s=1, b=1} \frac{\Gamma_1(1-P_s)^2}{\Gamma_1(2-2P_s)} = -\sqrt{\frac{2}{\pi}}. \quad (52)$$

All in all we obtain

$$\operatorname{Res}_{P_s=1, b=1} \mathcal{I}(P_s) = -\frac{1}{2\pi}. \quad (53)$$

Something very peculiar happens. If we were to close the contour on the right half plane so that we pick up the poles at $P_s = 1/2, 1$, we would obtain

$$-2\pi \frac{i}{i} \times \left(\operatorname{Res}_{P_s=1/2, b=1} \mathcal{I}(P_s) + \operatorname{Res}_{P_s=1, b=1} \mathcal{I}(P_s) \right) = \frac{4-\sqrt{2}}{4} \quad (54)$$

Remark in particular that

$$\sin^2(\pi/8) = -2\pi \frac{i}{i} \times \left(\operatorname{Res}_{P_s=1/2, b=1} \mathcal{I}(P_s) + \frac{1}{2} \operatorname{Res}_{P_s=1, b=1} \mathcal{I}(P_s) \right) \quad (55)$$

The fact that we have to take only half of the residue at $P_s = 1$ is reminiscent of a variant of the Sokhotski-Plemelj formula. Let's suppose we close the contour on the right half plane with a rectangle shape avoiding the pole at $P_s = 1$ via a small semi-circle ϵ . We also suppose that the top and bottom contributions vanish. We thus obtain

$$\begin{aligned} \int_{i\mathbb{R}} \frac{dP_s}{i} \mathcal{I}(P_s) &= -2\pi \left(\operatorname{Res}_{P_s=1/2, b=1} \mathcal{I}(P_s) \right) + \int_{C_\epsilon \sqcup i\mathbb{R}+1} \frac{dP_s}{i} \mathcal{I}(P_s). \\ &= -2\pi \left(\operatorname{Res}_{P_s=1/2, b=1} \mathcal{I}(P_s) \right) + \operatorname{PV} \int_{i\mathbb{R}+1} \frac{dP_s}{i} \mathcal{I}(P_s) - \pi \left(\operatorname{Res}_{P_s=1, b=1} \mathcal{I}(P_s) \right). \end{aligned} \quad (56)$$

Hence, providing the symmetric principal value of the integral at $P_s = 1$ vanishes, we obtain the desired result. There remains two things to be checked.

- The contribution of the top and bottom parts of the contour vanish.
- The symmetric principal value of the integral at $P_s = 1$ vanishes. This could be a consequence of a symmetry¹

$$\mathcal{I}(1+ix) = -\mathcal{I}(1-ix). \quad (57)$$

¹This is a kind of oddness property: recall that the symmetric principal value of $1/x$ at 0 vanishes because the function is odd.

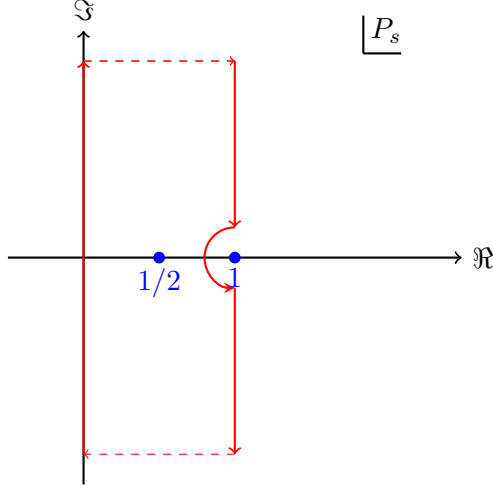


Figure 1: Contour deformation for the integral in the pentagon identity. In blue, the relevant residues. The dashed red lines are supposed to be pushed at ∞ where the integrand vanishes.

To show the first point, we can use the asymptotics of the Barnes G function and the relation between the double Gamma function and the Barnes G function for $b = 1$

$$\begin{aligned} \log(G(z+1)) &= \frac{z^2}{2} \log(z) - \frac{3z^2}{4} + \frac{z}{2} \log(2\pi) + \mathcal{O}(\log|z|), \\ \Gamma_1(z) &= \frac{(2\pi)^{\frac{z-1}{2}}}{G(z)}. \end{aligned} \quad (58)$$

I found

$$|\Gamma_1(x+iy)| = \exp \left[\frac{y^2}{2} \log|y| - \frac{3y^2}{4} + \frac{\pi}{2}(x-1)|y| + \mathcal{O}(\log|y|) \right]. \quad (59)$$

We then write $P_s = \tilde{x} + ip_s$ with $\tilde{x} \in [0, 1]$. Remembering the complete expression of the integrand, we find that the integrand behaves as (independently of the values of $\tilde{x}$)

$$\mathcal{I}(P_s) \sim K \times \frac{e^{\frac{3\pi}{2}|p_s|}}{p_s} \times e^{-4\pi|p_s|} \xrightarrow{p_s \rightarrow \pm\infty} 0, \quad (60)$$

where K is just an irrelevant prefactor independent of P_s . This concludes the part regarding the vanishing of the top and bottom parts of the contour.

We now turn to the second point. We will actually prove a stronger statement

$$\mathcal{I}(2 - P_s) = -\mathcal{I}(P_s) \quad \forall P_s \in \mathbb{C}. \quad (61)$$

This is an immediate consequence of the following shift relation of the structure constants

$$\frac{C_0\left(\frac{1}{4}, \frac{1}{4}, 2-P\right)}{C_0\left(\frac{1}{4}, \frac{1}{4}, P\right)} = -\frac{2-P}{P} 16^{4(1-P)}. \quad (62)$$

This can be easily checked using Gamma function identities (shift relation of double Gamma function, Euler reflection formula, and Legendre duplication formula).

Idempotency relation Finally, the idempotency relation reads

$$\int_{\mathbb{R}} dp \mathbb{F}_{P_s, ip}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \mathbb{F}_{iP, P_s'}^{(b)} \begin{bmatrix} P_3 & P_4 \\ P_2 & P_1 \end{bmatrix} = \frac{1}{2} (\delta(P_s - P_s') + \delta(P_s + P_s')). \quad (63)$$

Modular kernel We start by defining the necessary expressions for the quantum modular polynomial. Since you give explicit expressions in the article, I will directly use these expressions. The quantum modular determinant reads

$$\mathcal{D}^{(m)} = 1 - \left(\frac{s_{mn} \left(\frac{P_0}{2} - \frac{m+n}{4s} \right)}{s_{mn}(P_t) s_{mn}(P_s)} \right)^2, \quad (64)$$

while the roots are

$$z_{\pm}^{(m)} = \frac{s_{mn}(P_s)}{s_{mn} \left(\frac{P_0}{2} - P_s - \frac{m+n}{4s} \right)} \left[(-1)^{mn} c_{mn}(P_t) \mp i s_{mn}(P_t) \sqrt{\mathcal{D}^{(m)}} \right]. \quad (65)$$

We define the $\pm$ kernels whose expressions for rational charge are given in your paper Ioannis <https://arxiv.org/pdf/2512.03172>.

$$\mathbb{M}_{P_s, P_t}^{(\pm)}[P_0] = \pm \frac{M_b(P_0|P_s, P_t)}{2i \sin(2\pi s P_t) \sin(2\pi s P_s)} \frac{1}{\sqrt{\mathcal{D}^{(m)}}} \sum_{k=0}^{s^2-1} \mathcal{M}_b \left(\frac{\log z_{\pm}^{(m)}}{2\pi i s^2} + \frac{k}{s^2} \right). \quad (66)$$

We have

$$\mathcal{M}_b(u) = \frac{\text{Numerator}(\mathcal{M}_b)}{\text{Denominator}(\mathcal{M}_b)} \quad (67)$$

with

$$\begin{aligned} \text{Numerator}(\mathcal{M}_b) &= e^{-4\pi i s P_s u} \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2} \left(\frac{2P_t}{s} + \frac{P_0}{s} \right) \right) \\ &\times \tilde{G}_{m,n} \left(u - 1 + \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2} \left(-\frac{2P_t}{s} + \frac{P_0}{s} \right) \right) \end{aligned} \quad (68)$$

and

$$\begin{aligned} \text{Denominator}(\mathcal{M}_b) &= \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2} \left(\frac{2P_t}{s} - \frac{P_0}{s} \right) \right) \\ &\times \tilde{G}_{m,n} \left(u - \frac{m^{-1} + n^{-1}}{4} - \frac{1}{2} \left(-\frac{2P_t}{s} - \frac{P_0}{s} \right) \right) \end{aligned} \quad (69)$$

Next, we also have

$$M_b(P_0|P_s, P_t) = \frac{\text{Numerator}(\mathcal{M}_b)}{\text{Denominator}(\mathcal{M}_b)}, \quad (70)$$

with

$$\begin{aligned} \text{Numerator}(\mathcal{M}_b) &= s(2\pi)^{sP_0-m-n} \mathcal{N}_{m,n}(P_s) \rho_0^{(b)}(P_t) \\ &\times G_{m,n} \left(m^{-1} + n^{-1} + \frac{2P_t}{s} \right) G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} - \frac{P_0 + 2P_s}{s} \right) \\ &\times G_{m,n} \left(m^{-1} + n^{-1} - \frac{2P_t}{s} \right) G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} - \frac{P_0 - 2P_s}{s} \right), \end{aligned} \quad (71)$$

with $\mathcal{N}_{n,m} = (-1)^{nm} (2\pi)^{nm} s^{-2} e^{2\pi i s P_s}$ and $\rho_0^{(b)}$ is the Plancherel measure

$$\rho_0^{(b)}(P_t) = -4\sqrt{2} \sin(2\pi b P_t) \sin(2\pi b^{-1} P_t).$$

Finally the denominator reads

$$\begin{aligned} \text{Denominator}(\mathcal{M}_b) &= 2\tilde{G}_{m,n} \left(\frac{-P_0}{s} \right) \\ &\times G_{m,n} \left(m^{-1} + n^{-1} + \frac{2P_s}{s} \right) G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} - \frac{P_0 + 2P_t}{s} \right) \\ &\times G_{m,n} \left(m^{-1} + n^{-1} - \frac{2P_s}{s} \right) G_{m,n} \left(\frac{m^{-1} + n^{-1}}{2} - \frac{P_0 - 2P_t}{s} \right). \end{aligned} \quad (72)$$

Consistency checks for the modular kernel Before checking the idempotency relation and the torus two point function consistency condition, we will start by some trivial checks which will make sure that we implemented the correct definition. First, the sum defined in the $\pm$ kernels is single-valued and thus does not depend on the branch of the logarithm i.e

$$\sum_{k=0}^{s^2-1} \mathcal{M}_b \left(\frac{\log z_{\pm}^{(f)}}{2\pi i s^2} + \frac{k}{s^2} \right) = \sum_{k=0}^{s^2-1} \mathcal{M}_b \left(\frac{\log z_{\pm}^{(f)}}{2\pi i s^2} + \frac{k+\ell}{s^2} \right), \quad (73)$$

for all ℓ in $\mathbb{Z}$. A second non trivial check is the identity limit of the Teschner-Vartanov modular kernel precisely,

$$\mathbb{M}_{P_s, P_t}^{(TV)} \left[P_0 = -\frac{b+b^{-1}}{2} \right] = \sqrt{2} \cos(4\pi P_s P_t). \quad (74)$$

It captures modular covariance of the vacuum representation. Finally, we would like to check the reflection properties of the $\mathbb{M}_{P_s, P_t}^{(\pm)} [P_0]$ kernels. They have to transform into one another under reflection of both internal momenta, precisely,

$$\begin{aligned} \mathbb{M}_{P_s, P_t}^{(\pm)} [P_0] &\xrightarrow{P_s \rightarrow -P_s} \mathbb{M}_{P_s, P_t}^{(\mp)} [P_0] \\ \mathbb{M}_{P_s, P_t}^{(\pm)} [P_0] &\xrightarrow{P_t \rightarrow -P_t} \mathbb{M}_{P_s, P_t}^{(\mp)} [P_0]. \end{aligned} \quad (75)$$

Instead of checking the idempotency relation for the $\pm$ kernels, we will check the following relation which is a consequence of the idempotency relation

$$\mathbb{M}_{P_1, P'} [P_0] = \frac{\frac{C_0(P', P', P_0)}{B(P')}}{\frac{C_0(P_1, P_1, P_0)}{B(P_1)}} \mathbb{M}_{P', P_1} [P_0]. \quad (76)$$

We did not specify if this concerns the $\pm$ kernels or the TV kernel because we would like to check it for both of them.

Consistency checks involving the Fusion and the modular kernels Before turning to the most difficult equation which is consistency of the torus two point functions, we have the non-rational Verlinde formula presented in your paper Ioannis.

$$\mathbb{F}_{P_1, P_3}^{(b)} \begin{bmatrix} P_0 & P_3 \\ P_1 & P_2 \end{bmatrix} = \int_{\mathbb{R}} dp \frac{\mathbb{M}_{P_1, ip} [P_0] \mathbb{M}_{P_2, ip} [\mathbb{I}] \mathbb{M}_{ip, P_3} [P_0]}{\mathbb{M}_{\mathbb{I}, ip} [\mathbb{I}]}. \quad (77)$$

Finally, we have the consistency condition for the torus two point function which reads

$$\begin{aligned} \mathbb{M}_{P_1, P_2} [P_3] &\int_{\mathbb{R}} dp_4 \mathbb{F}_{P_3, ip_4} \begin{bmatrix} P'_0 & P_2 \\ P_0 & P_2 \end{bmatrix} e^{\pi i (2h_4 - 2h_2 + \frac{h_3}{2})} \mathbb{F}_{ip_4, P_5} \begin{bmatrix} P'_0 & P_0 \\ P_2 & P_2 \end{bmatrix} \\ &= \int_{\mathbb{R}} dp_6 \mathbb{F}_{P_3, ip_6} \begin{bmatrix} P_0 & P_1 \\ P'_0 & P_1 \end{bmatrix} \mathbb{F}_{P_1, P_5} \begin{bmatrix} P'_0 & P_0 \\ ip_6 & ip_6 \end{bmatrix} e^{\pi i (h_0 + h'_0 - \frac{h_5}{2})} \mathbb{M}_{ip_6, P_2} [P_5]. \end{aligned} \quad (78)$$

As the LHS integral is not absolutely convergent, we will shift the contour as

$$\begin{aligned} \mathbb{M}_{P_1, P_2} [P_3] &\int_{\mathbb{R}} d(p_4 e^{i\pi/12}) \mathbb{F}_{P_3, ip_4 e^{i\pi/12}} \begin{bmatrix} P'_0 & P_2 \\ P_0 & P_2 \end{bmatrix} e^{\pi i (2h_4(p_4 e^{i\pi/12}) - 2h_2 + \frac{h_3}{2})} \mathbb{F}_{ip_4 e^{i\pi/12}, P_5} \begin{bmatrix} P'_0 & P_0 \\ P_2 & P_2 \end{bmatrix} \\ &= \int_{\mathbb{R}} dp_6 \mathbb{F}_{P_3, ip_6} \begin{bmatrix} P_0 & P_1 \\ P'_0 & P_1 \end{bmatrix} \mathbb{F}_{P_1, P_5} \begin{bmatrix} P'_0 & P_0 \\ ip_6 & ip_6 \end{bmatrix} e^{\pi i (h_0 + h'_0 - \frac{h_5}{2})} \mathbb{M}_{ip_6, P_2} [P_5]. \end{aligned} \quad (79)$$

2.4 Timelike expressions

Virasoro Wick rotation The kernels are obtained through the Virasoro Wick rotation of the spacelike kernels. It reads

$$\begin{aligned}\hat{\mathbb{M}}_{P_s, P_t}^{(\pm)}[P_0] &= \mp i \frac{P_t}{P_s} \mathbb{M}_{iP_t, iP_s}[iP_0] \\ \hat{\mathbb{F}}_{P_s, P_t}^{(\pm)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} &= \mp i \frac{P_t}{P_s} \mathbb{F}_{iP_t, iP_s}^{(\pm)} \begin{bmatrix} iP_2 & iP_1 \\ iP_3 & iP_4 \end{bmatrix}\end{aligned}\quad (80)$$

Note that the value of b remains formally unchanged that is to obtain the timelike kernel at $c = 1$, it suffices to apply the prescription (80) with $b = 1$ for evaluation of the RHS.

Consistency checks for the fusion kernel A simple but non trivial property of the kernels is their transformation rules under reflection of internal momenta. Basically, $\hat{\mathbb{F}}^{(\pm)}$ should get exchanged under reflection of the internal momenta eg $P_s \rightarrow -P_s$. Before checking the pentagon equation for the spacelike kernels, we will first verify a relation that was written by Ioannis and Sylvain in the paper concerning the $c = 1$ fusion kernels. Precisely, the relation reads

$$\hat{\mathbb{F}}_{P_s, P_t}^{(\pm)} \begin{bmatrix} i/4 & i/4 \\ i/4 & i/4 \end{bmatrix} = 16^{P_t^2 - P_s^2} e^{\mp 2\pi i P_s P_t} . \quad (81)$$

We also have the analog of the mixing symmetry for the timelike fusion kernels which reads

$$\hat{\mathbb{F}}_{P_s, P_t}^{(\eta)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} = \frac{\Psi_{P_3, P_t}[P_2]}{\Psi_{P_s, P_1}[P_2]} \hat{\mathbb{F}}_{P_3, P_1}^{(\eta)} \begin{bmatrix} P_2 & P_s \\ P_t & P_4 \end{bmatrix} , \quad (82)$$

where we define

$$\Psi_{P_1, P_2}[P_3] = \frac{P_2}{P_1} \hat{\rho}_0(P_1) \hat{C}_0(P_1, P_2, P_3) . \quad (83)$$

After checking that we are ready to implement the pentagon identity for the spacelike kernels. In order to test if it satisfied also for $\hat{\mathbb{F}}^{(\pm)}$ and only for the analog of the TV kernel, we will implement the following:

$$\begin{aligned}\int_{\mathbb{R}} dp_s \hat{\mathbb{F}}_{P_q, P_4} \begin{bmatrix} ip_s & P_3 \\ P_u & P_5 \end{bmatrix} \hat{\mathbb{F}}_{P_o, ip_s} \begin{bmatrix} P_1 & P_2 \\ P_u & P_q \end{bmatrix} \hat{\mathbb{F}}_{ip_s, P_t}^{(\text{phys.})} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \\ = \hat{\mathbb{F}}_{P_o, P_4} \begin{bmatrix} P_1 & P_t \\ P_u & P_5 \end{bmatrix} \hat{\mathbb{F}}_{P_q, P_t} \begin{bmatrix} P_2 & P_3 \\ P_o & P_5 \end{bmatrix}\end{aligned}\quad (84)$$

where $\hat{\mathbb{F}}$ denotes a generic kernel (it could be $\mathbb{F}^{(\pm)}$ or the physical one $\hat{\mathbb{F}}^{(\text{phys.})}$ or something else that has the same arguments). Finally, the idempency relation reads

$$\int_{\mathbb{R}} dp \hat{\mathbb{F}}_{P_s, ip}^{(b)} \begin{bmatrix} P_2 & P_3 \\ P_1 & P_4 \end{bmatrix} \hat{\mathbb{F}}_{ip, P'_s}^{(b)} \begin{bmatrix} P_3 & P_4 \\ P_2 & P_1 \end{bmatrix} = \frac{1}{2} (\delta(P_s - P'_s) + \delta(P_s + P'_s)) . \quad (85)$$

3 Numerical Packages/conventions

Since the only library that I found that does implement Barnes G-function is mpmath, we will use mpmath for the special functions.

- Error estimates / tolerances (SciPy quad, etc.)
- Complex integration conventions (parametrized contours, branch choices)

4 Explanations/Workflow

I invite you also to check the end of the workflow since there are a bunch of checks that everything work correctly so far.

Pentagon equation/spacelike The values of P 's and m, n will be our global variables as well as the cutoff Λ for the integration. We start by the packages and we define a dataclass for our global variables. Also, we define a DEMO mod which is True for the demo tests and false for the final checks which we will perform at the very end.

```
#Packages
from dataclasses import dataclass
from mpmath import mp, barnesg
import numpy as np
import matplotlib.pyplot as plt

# ----- Global settings -----
mp.dps = 20 #be careful with the precision, if the precision is not
            enough you might raise "Zero Division error"
DEMO = True # True for Demo mode

# ----- Parameters container -----
@dataclass(frozen=True)
class ParametersPentagon:
    Po: mp.mpc
    Pq: mp.mpc
    P1: mp.mpc
    P2: mp.mpc
    P3: mp.mpc
    P4: mp.mpc
    P5: mp.mpc
    Pt: mp.mpc
    Pu: mp.mpc
@dataclass(frozen=True)
class ParametersIdempotency:
    Ps: mp.mpc
    Psprime: mp.mpc
    P1: mp.mpc
    P2: mp.mpc
    P3: mp.mpc
    P4: mp.mpc

@dataclass
class Theory:
    m: int
    n: int
    Lambda: mp.mpf = mp.mpf(3) # cutoff (for later integration)

    @property
    def b(self):
        return mp.sqrt(mp.mpf(self.m) / mp.mpf(self.n))

    @property
    def s(self):
        return mp.sqrt(mp.mpf(self.m) * mp.mpf(self.n))
```

```
# ----- Demo / main params -----
if DEMO:
    parPenta = ParametersPentagon(
        Po=mp.mpc(0.001+0.1j), Pq=mp.mpc(0.001+0.5j),
        P1=mp.mpc(0.001+0.2j), P2=mp.mpc(0.001+0.6j),
        P3=mp.mpc(0.001+0.3j), P4=mp.mpc(0.001+0.7j),
        P5=mp.mpc(0.001+0.4j), Pt=mp.mpc(0.001+0.8j),
        Pu=mp.mpc(0.001+0.9j)
    )
    parIdem = ParametersIdempotency(
        Ps=mp.mpc(0.001+0.1j), Psprime=mp.mpc(0.001+3j),
        P1=mp.mpc(0.001+0.2j), P2=mp.mpc(0.001+0.6j),
        P3=mp.mpc(0.001+0.3j), P4=mp.mpc(0.001+0.7j),
    )
    theo=Theory(m=1,n=1)
else:
    parPenta = ParametersPentagon(
        Po=mp.mpc(0.0+0.1j), Pq=mp.mpc(0.0+0.2j),
        P1=mp.mpc(0.0+0.3j), P2=mp.mpc(0.0+0.4j),
        P3=mp.mpc(0.0+0.5j), P4=mp.mpc(0.0+0.6j),
        P5=mp.mpc(0.0+0.7j), Pt=mp.mpc(0.0+0.8j),
        Pu=mp.mpc(0.001+0.9j)
    )
    theo=Theory(m=1,n=1)
```

In addition to these preliminary data that we defined, we also introduce some basics modules that could be useful if we want to plot the phase or the modulus of a given function.

```
# =====
# Useful modules to plot a bunch of different things
# =====

def plot_phase2D(func, x_min=-2, x_max=2, y_min=-2, y_max=2, resolution
=100):

    x = np.linspace(x_min, x_max, resolution)
    y = np.linspace(y_min, y_max, resolution)
    X, Y = np.meshgrid(x, y)
    Z = np.zeros_like(X, dtype=np.float64)

    for i in range(resolution):
        for j in range(resolution):
            z_val = func(mp.mpc(X[i, j], Y[i, j]))
            Z[i, j] = mp.arg(z_val)

    plt.figure(figsize=(8, 6))
    plt.contourf(X, Y, Z, levels=100, cmap='hsv')
    plt.colorbar(label='Phase (radians)')
    plt.title('Phase of the function in the complex plane')
    plt.xlabel('Real part')
    plt.ylabel('Imaginary part')
    plt.xlim(x_min, x_max)
    plt.ylim(y_min, y_max)
    plt.show()

def plot_module1D(func, name: str, x_min=-5, x_max=5, resolution=300):
    x = np.linspace(x_min, x_max, resolution)
    y = np.zeros(resolution, dtype=float)
```

```

for i in range(resolution):
    z_val = func(mp.mpc(x[i]))
    y[i] = float(abs(z_val))

plt.figure(figsize=(8, 6))
plt.plot(x, y)
plt.title('Modulus of the function')
plt.xlabel('p')
plt.ylabel(name)
plt.xlim(x_min, x_max)
plt.grid(True)
plt.show()

def plot_real1D(func, name: str, x_min=-5, x_max=5, resolution=300):
    x = np.linspace(x_min, x_max, resolution)
    y = np.zeros(resolution, dtype=float)

    for i in range(resolution):
        z_val = func(mp.mpc(x[i]))
        y[i] = float(mp.re(z_val))

    plt.figure(figsize=(8, 6))
    plt.plot(x, y)
    plt.title('Real part of the function')
    plt.xlabel('p')
    plt.ylabel(name)
    plt.xlim(x_min, x_max)
    plt.grid(True)
    plt.show()

```

We then define the special functions $\tilde{G}$, $G_{m,n}$ and $\tilde{G}_{m,n}$ (cf (6), (4) and (7)). These functions have some properties that we can check numerically (cf (9) and (10)). We will also check the shift relation for $G_{m,n}$ (cf (5)). These checks will be implemented at the end of the workflow.

```

# =====
# Barnes G products
# =====

def Gmn(Theory: Theory, z: complex):
    m, n = Theory.m, Theory.n
    z = mp.mpc(z)

    prod = mp.mpc(1)
    m_mp = mp.mpf(m)
    n_mp = mp.mpf(n)

    for k in range(m):
        kshift = mp.mpf(k) / m_mp
        for l in range(n):
            prod *= barnesg(z + kshift + mp.mpf(l) / n_mp)
    return prod

def Gtilde(z: complex):
    """
    \tilde{G}(z) = G(1+z)/G(1-z)
    """
    z = mp.mpc(z)
    return barnesg(1 + z) / barnesg(1 - z)

```

```

def Gtilde_mn(Theory: Theory, z:complex):
    m, n = Theory.m, Theory.n
    z = mp.mpc(z)

    prod = mp.mpc(1)
    m_mp = mp.mpf(m)
    n_mp = mp.mpf(n)

    half = mp.mpf(1) / 2
    for k in range(m):
        kshift = mp.mpf(k) / m_mp + half / m_mp
        for l in range(n):
            shift = kshift + mp.mpf(l) / n_mp + half / n_mp
            prod *= Gtilde(z + shift - 1)
    return prod

```

We will then turn on to the expression of the quantum modular polynomial and the determinant. First, we implement the trigonometric functions c_{mn} and s_{mn} as well as the shifted momenta a_i .

```

# =====
# Trig helpers + a(z)
# =====

def smn(Theory: Theory, z:complex):
    return mp.sin(2 * mp.pi * Theory.s * mp.mpc(z))

def cmn(Theory: Theory, z:complex):
    return mp.cos(2 * mp.pi * Theory.s * mp.mpc(z))

def a(Theory: Theory, z:complex):
    # a_i = P_i - (m+n)/(2s) in your notation; here: a(Theory, z) = z -
    # (m+n)/(2s)
    return mp.mpc(z) - (mp.mpf(Theory.m + Theory.n) / (2 * Theory.s))

```

We start by the fusion. We first define all the relevant quantities for the quantum modular fusion polynomial

```

# =====
# =====
# FUSION KERNEL
# =====
# =====

# =====
# Quantum modular fusion polynomial
# =====

# Parameters of the polynomial
def alphaf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:
    complex, Ps:complex, Pt:complex):

    P1, P2, P3, P4, Ps, Pt = mp.mpc(P1), mp.mpc(P2), mp.mpc(P3), mp.mpc(
        P4), mp.mpc(Ps), mp.mpc(Pt)
    m, n = Theory.m, Theory.n
    s = Theory.s

```

```

m_inv = mp.mpf(1) / mp.mpf(m)
n_inv = mp.mpf(1) / mp.mpf(n)

pref_plus = 2 * mp.pi * 1j * s**2 * (m_inv + n_inv) / 4
pref_minus = 2 * mp.pi * 1j * s**2 * (-m_inv - n_inv) / 4

def E(pref, lin):
    return mp.e ** (pref + mp.pi * 1j * s * lin)

term1 = E(pref_plus, Ps + Pt - P1 - P2 - P3 - P4)
term2 = E(pref_plus, Ps + Pt + P1 + P2 + P3 + P4)
term3 = E(pref_plus, -Ps - Pt - P1 + P2 - P3 + P4)
term4 = E(pref_plus, -Ps - Pt + P1 - P2 + P3 - P4)

term5 = E(pref_minus, -Ps + Pt - P1 - P2 + P3 + P4)
term6 = E(pref_minus, -Ps + Pt + P1 + P2 - P3 - P4)
term7 = E(pref_minus, Ps - Pt - P1 + P2 + P3 - P4)
term8 = E(pref_minus, Ps - Pt + P1 - P2 - P3 + P4)

return term1 + term2 + term3 + term4 - term5 - term6 - term7 - term8

def betaf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:complex
, Ps:complex, Pt:complex):
    return 4*(smn(Theory,P1)*smn(Theory,P3)+smn(Theory,P2)*smn(Theory,P4
)+smn(Theory,Ps)*smn(Theory,Pt))

def gammaf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:
complex, Ps:complex, Pt:complex):
    P1, P2, P3, P4, Ps, Pt = mp.mpc(P1), mp.mpc(P2), mp.mpc(P3), mp.mpc(
        P4), mp.mpc(Ps), mp.mpc(Pt)
    m, n = Theory.m, Theory.n
    s = Theory.s

    m_inv = mp.mpf(1) / mp.mpf(m)
    n_inv = mp.mpf(1) / mp.mpf(n)

    pref_plus = 2 * mp.pi * 1j * s**2 * (m_inv + n_inv) / 4
    pref_minus = 2 * mp.pi * 1j * s**2 * (-m_inv - n_inv) / 4

    def E(pref, lin):
        return mp.e ** (-pref - mp.pi * 1j * s * lin)

    term1 = E(pref_plus, Ps + Pt - P1 - P2 - P3 - P4)
    term2 = E(pref_plus, Ps + Pt + P1 + P2 + P3 + P4)
    term3 = E(pref_plus, -Ps - Pt - P1 + P2 - P3 + P4)
    term4 = E(pref_plus, -Ps - Pt + P1 - P2 + P3 - P4)

    term5 = E(pref_minus, -Ps + Pt - P1 - P2 + P3 + P4)
    term6 = E(pref_minus, -Ps + Pt + P1 + P2 - P3 - P4)
    term7 = E(pref_minus, Ps - Pt - P1 + P2 + P3 - P4)
    term8 = E(pref_minus, Ps - Pt + P1 - P2 - P3 + P4)

    return term1 + term2 + term3 + term4 - term5 - term6 - term7 - term8

#Determinant of the quantum modular polynomial
def Gram_matrixf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:
complex, Ps:complex, Pt:complex):

```

```

P1, P2, P3, P4, Ps, Pt = mp.mpc(P1), mp.mpc(P2), mp.mpc(P3), mp.mpc(P4), mp.mpc(Ps), mp.mpc(Pt)
m, n = Theory.m, Theory.n
s = Theory.s
G = mp.zeros(4, 4)
for i in range(4):
    G[i, i] = 1
    G[0,1]=-cmn(Theory, a(Theory, P2))
    G[0,2]=-cmn(Theory, a(Theory, P3))
    G[0,3]=cmn(Theory, a(Theory, Ps))
    G[1,0]=-cmn(Theory, a(Theory, P2))
    G[1,2]=cmn(Theory, a(Theory, Pt))
    G[1,3]=-cmn(Theory, a(Theory, P1))
    G[2,0]=-cmn(Theory, a(Theory, P3))
    G[2,1]=cmn(Theory, a(Theory, Pt))
    G[2,3]=-cmn(Theory, a(Theory, P4))
    G[3,0]=cmn(Theory, a(Theory, Ps))
    G[3,1]=-cmn(Theory, a(Theory, P1))
    G[3,2]=-cmn(Theory, a(Theory, P4))
return G

def Gram_determinantf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:complex, Ps:complex, Pt:complex):
    G = Gram_matrixf(Theory, P1, P2, P3, P4, Ps, Pt)
    return mp.det(G)

def quantum_determinantf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:complex, Ps:complex, Pt:complex):
    return -4*Gram_determinantf(Theory, P1, P2, P3, P4, Ps, Pt)/(smn(Theory, Ps)**2*smn(Theory, Pt)**2)

# roots of the polynomial
def rootsf(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:complex, Ps:complex, Pt:complex):
    beta_val = betaf(Theory, P1, P2, P3, P4, Ps, Pt)
    alpha_val = alphaf(Theory, P1, P2, P3, P4, Ps, Pt)

    z_plus = (-beta_val +2j*smn(Theory, Ps)*smn(Theory, Pt)*mp.sqrt(quantum_determinantf(Theory, P1, P2, P3, P4, Ps, Pt))) / (2 * alpha_val)
    z_minus = (-beta_val -2j*smn(Theory, Ps)*smn(Theory, Pt)*mp.sqrt(quantum_determinantf(Theory, P1, P2, P3, P4, Ps, Pt))) / (2 * alpha_val)

    return z_plus, z_minus

```

We then define the $\pm$ kernels as well as the (TV) kernel. The function "CurlyF" corresponds to $\mathcal{F}_b$ while "F" corresponds to F_b . These are the elementary blocks from which the kernels are built (see (66)). The module "Kernels" does implement the kernels. Note that this module contains ϵ as an argument. $\epsilon = \pm 1$ corresponds to the kernels $F^\pm$ while $\epsilon = 0$ implements the TV kernel.

```

# =====
# Fusion Kernels
# =====

# These are the elementary building blocks of the kernels
def CurlyF(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:

```

```

complex, Ps:complex, Pt:complex, z:complex):
    P1, P2, P3, P4, Ps, Pt, z = mp.mpc(P1), mp.mpc(P2), mp.mpc(P3), mp.
        mpc(P4), mp.mpc(Ps), mp.mpc(Pt), mp.mpc(z)
    m, n = Theory.m, Theory.n
    s = Theory.s

    term1num = Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (-Ps+Pt-P1-P2+P3+P4)/(2*s)$ )
    term2num = Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (-Ps+Pt+P1+P2-P3-P4)/(2*s)$ )
    term3num = Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (Ps-Pt-P1+P2+P3-P4)/(2*s)$ )
    term4num = Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (Ps-Pt+P1-P2-P3+P4)/(2*s)$ )
    Numerator = term1num*term2num*term3num*term4num

    term1den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (Ps+Pt-P1-P2-P3-P4)/(2*s)$ )
    term2den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (Ps+Pt+P1+P2+P3+P4)/(2*s)$ )
    term3den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (-Ps-Pt-P1+P2-P3+P4)/(2*s)$ )
    term4den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (-Ps-Pt+P1-P2+P3-P4)/(2*s)$ )
    Denominator = term1den*term2den*term3den*term4den

    return Numerator/Denominator

def F(Theory: Theory, P1:complex, P2:complex, P3:complex, P4:complex, Ps:
    complex, Pt:complex):
    P1, P2, P3, P4, Ps, Pt = mp.mpc(P1), mp.mpc(P2), mp.mpc(P3), mp.mpc(
        P4), mp.mpc(Ps), mp.mpc(Pt)
    m, n = Theory.m, Theory.n
    s = Theory.s

    term0num=1j*(2*mp.pi)**(4*s**2-(m+n))*Gmn(Theory, 2*Pt/s)*Gmn(Theory
        , -2*Pt/s)
    term1num=Gmn(Theory, (1/m + 1/n)/2 + (P1-P2+Ps)/s)
    term2num=Gmn(Theory, (1/m + 1/n)/2 + (-P1+P2+Ps)/s)
    term3num=Gmn(Theory, (1/m + 1/n)/2 + (P1+P2-Ps)/s)
    term4num=Gmn(Theory, (1/m + 1/n)/2 + (-P1-P2-Ps)/s)
    term5num=Gmn(Theory, (1/m + 1/n)/2 + (P3-P4+Ps)/s)
    term6num=Gmn(Theory, (1/m + 1/n)/2 + (-P3+P4+Ps)/s)
    term7num=Gmn(Theory, (1/m + 1/n)/2 + (P3+P4-Ps)/s)
    term8num=Gmn(Theory, (1/m + 1/n)/2 + (-P3-P4-Ps)/s)
    Numerator = term0num*term1num*term2num*term3num*term4num*term5num*
        term6num*term7num*term8num

    term0den=s*Gmn(Theory, 1/m + 1/n + 2*Ps/s)*Gmn(Theory, 1/m + 1/n - 2*Ps/
        s)
    term1den=Gmn(Theory, (1/m + 1/n)/2 + (P2+P3+Pt)/s)
    term2den=Gmn(Theory, (1/m + 1/n)/2 + (P2-P3-Pt)/s)
    term3den=Gmn(Theory, (1/m + 1/n)/2 + (-P2-P3+Pt)/s)
    term4den=Gmn(Theory, (1/m + 1/n)/2 + (-P2+P3-Pt)/s)
    term5den=Gmn(Theory, (1/m + 1/n)/2 + (P1+P4+Pt)/s)
    term6den=Gmn(Theory, (1/m + 1/n)/2 + (P1-P4-Pt)/s)
    term7den=Gmn(Theory, (1/m + 1/n)/2 + (-P1-P4+Pt)/s)
    term8den=Gmn(Theory, (1/m + 1/n)/2 + (-P1+P4-Pt)/s)

```

```

        Denominator = term0den*term1den*term2den*term3den*term4den*term5den*
            term6den*term7den*term8den

    return Numerator/Denominator

#These are the plus/minus kernels/TV kernels
def Kernelsf(Theory: Theory, P1: complex, P2: complex, P3: complex, P4:
    complex, Ps: complex, Pt: complex, epsilon: int):

    # epsilon =\pm are the elementary kernels while epsilon=0 is the
        Teschner-Vartanov kernel
    s = Theory.s
    N = Theory.m * Theory.n # this is s^2 as an integer

    zplus, zminus = rootsf(Theory, P1, P2, P3, P4, Ps, Pt)

    prefactor = (
        F(Theory, P1, P2, P3, P4, Ps, Pt)
        / (2j * smn(Theory, Pt) * smn(Theory, Ps) * mp.sqrt(
            quantum_determinantf(Theory, P1, P2, P3, P4, Ps, Pt)))
    )

    bigsum = mp.mpc(0)

    if epsilon == 1:
        u0 = mp.log(zplus) / (2j * mp.pi * s**2)
        for k in range(N):
            bigsum += CurlyF(Theory, P1, P2, P3, P4, Ps, Pt, u0 + mp.mpf
                (k) / N)
        return prefactor * bigsum

    elif epsilon == -1:
        u0 = mp.log(zminus) / (2j * mp.pi * s**2)
        for k in range(N):
            bigsum += CurlyF(Theory, P1, P2, P3, P4, Ps, Pt, u0 + mp.mpf
                (k) / N)
        return -prefactor * bigsum

    elif epsilon ==0:
        return (Kernelsf(Theory, P1, P2, P3, P4,
            Ps, Pt,1)+Kernelsf(Theory, P1, P2, P3, P4,
            Ps, Pt,-1))/2

    else:
        raise ValueError("epsilon must be either 0,1 or -1")

```

All the modules that we define in what follows correspond to identities of the fusion kernels that we check. These identities are:

- The reflection property of the fusion kernels
- The mixing symmetry (see (33))
- The identity derived by Ioannis and Sylvain for $c=25$ (see (34)).
- The pentaton identity (see (84)).
- The idempotency relation (see (85)).

```

# =====
# Identities to be checked for the fusion kernel

```

```

# =====
def mixingsymmetrieskernelsf(Theory: Theory, P1: complex, P2: complex,
    P3: complex, P4: complex,
        Ps: complex, Pt: complex, epsilon: int):
    b = Theory.b
    vacuummomentum=(b + 1/b)/2 +0.00001j #for this momentum = P<1,1>,
        there is a "fake pole" numerically! For this reason, I shifted a
        bit by giving them a small imaginary part
    return Kernelsf(Theory,P2,P2,P3,P3,vacuummomentum,Pt,0)*Kernelsf(
        Theory,Pt,P2,Ps,P4,P3,P1,epsilon)/(Kernelsf(Theory,P2,P2,Ps,Ps,
            vacuummomentum,P1,0))

# This is the identity that Ionniss wrote down in his paper with Sylvain
def RibaultTsiaresspacelikef(Theory: Theory,Ps:complex, Pt:complex,
    epsilon:int):
    Ps, Pt =mp.mpc(Ps), mp.mpc(Pt)
    m, n = Theory.m, Theory.n
    if (m!=1 or n!=1):
        raise ValueError("The central charge is not 25")
    else:
        if (epsilon==1):
            return 1j*Pt*16**((Pt**2-Ps**2)*mp.exp(2j*mp.pi*Ps*Pt)/Ps
        elif (epsilon==-1):
            return -1j*Pt*16**((Pt**2-Ps**2)*mp.exp(-2j*mp.pi*Ps*Pt)/Ps
        else:
            raise ValueError("epsilon must be either 1 or -1")

# This returns the LHS and the RHS of the Pentagon identity
def Pentagon(Theory: Theory, Param: ParametersPentagon, epsilon: int):
    Po, Pq, P1, P2, P3, P4, P5, Pt, Pu = (
        Param.Po, Param.Pq, Param.P1, Param.P2,
        Param.P3, Param.P4, Param.P5, Param.Pt, Param.Pu
    )

    IntegrandLHS = lambda ps: (
        Kernelsf(Theory, Pu, 1j*(ps+0.0001j), P3, P5, Pq, P4, epsilon)
        * Kernelsf(Theory, Pu, P1, P2, Pq, Po, 1j*(ps+0.0001j), epsilon)
        * Kernelsf(Theory, P1, P2, P3, P4, 1j*(ps+0.0001j), Pt, 0)
    )

    LHS = mp.quad(IntegrandLHS, [-Theory.Lambda, Theory.Lambda])

    RHS = (
        Kernelsf(Theory, Pu, P1, Pt, P5, Po, P4, epsilon)
        * Kernelsf(Theory, Po, P2, P3, P5, Pq, Pt, epsilon)
    )

    return LHS, RHS

def Idempotencyf(Theory: Theory, Param:ParametersIdempotency, epsilon:
    int):
    Ps, Psprime, P1, P2, P3, P4 = (
        Param.Ps, Param.Psprime, Param.P1, Param.P2,
        Param.P3, Param.P4
    )
    Integrand = lambda p: (
        Kernelsf(Theory, P1, P2, P3, P4, Ps, 1j*(p+0.0001j), epsilon)

```

```

        * Kernelsf(Theory, P2, P3, P4, P1, 1j*(p+0.0001j), Psprime,
            epsilon)
    )

    LHS = mp.quad(Integrand, [-Theory.Lambda, Theory.Lambda])

    return LHS

```

We then define the modular kernels. We start by the quantum modular polynomial.

```

# =====
# =====
# Modular kernel
# =====
# =====

# =====
# Quantum modular modular polynomial
# =====

# quantum modular determinant D
def quantum_determinantm(Theory: Theory, P0:complex, Ps:complex, Pt:
    complex):
    m, n= Theory.m, Theory.n
    s= Theory.s
    return 1-(smn(Theory, P0/2 - (m+n)/(4*s))/(smn(Theory, Pt)*smn(
        Theory, Ps)))**2

# roots of the polynomial
def rootsm(Theory: Theory, P0:complex, Ps:complex, Pt:complex):
    m, n= Theory.m, Theory.n
    s= Theory.s
    N = m*n

    prefactor = smn(Theory,Ps)/smn(Theory,P0/2 -Ps -(m+n)/(4*s))

    plusrootfactor = (-1)**N*cmn(Theory,Pt)-1j*smn(Theory,Pt)*mp.sqrt(
        quantum_determinantm(Theory,P0,Ps,Pt))
    minusrootfactor = (-1)**N*cmn(Theory,Pt)+1j*smn(Theory,Pt)*mp.sqrt(
        quantum_determinantm(Theory,P0,Ps,Pt))

    z_plus= prefactor*plusrootfactor
    z_minus= prefactor*minusrootfactor

    return z_plus, z_minus

```

We then construct the modular kernels.

```

# =====
# Modular Kernels
# =====

# These are the elementary building blocks of the modular kernels
def CurlyM(Theory: Theory, P0:complex, Ps:complex, Pt:complex, z:complex
    ):
    P0, Ps, Pt, z = mp.mpc(P0), mp.mpc(Ps), mp.mpc(Pt), mp.mpc(z)
    m, n = Theory.m, Theory.n
    s = Theory.s

```

```

term1num = mp.exp(-4j*mp.pi*Ps*z*s)*Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (2*Pt+P0)/(2*s)$ )
term2num = Gtilde_mn(Theory, z  $-1+(1/m + 1/n)/4 - (-2*Pt+P0)/(2*s)$ )
Numerator = term1num*term2num

term1den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (2*Pt-P0)/(2*s)$ )
term2den = Gtilde_mn(Theory, z  $-(1/m + 1/n)/4 - (-2*Pt-P0)/(2*s)$ )
Denominator = term1den*term2den

return Numerator/Denominator

def M(Theory: Theory, P0:complex, Ps:complex, Pt:complex):
    P0, Ps, Pt = mp.mpc(P0), mp.mpc(Ps), mp.mpc(Pt)
    m, n = Theory.m, Theory.n
    s = Theory.s
    b = Theory.b
    N = m*n

    Nmn = (-1)**N * (2*mp.pi)**(2*N) * mp.exp(2j*mp.pi*s*Ps)/(s**2)
    Plancherel = -4*mp.sqrt(2)*mp.sin(2*mp.pi*b*Pt)*mp.sin(2*mp.pi*Pt/b)

    term0num=s*(2*mp.pi)**(s*P0-m-n) *Nmn*Plancherel
    term1num=Gmn(Theory,1/m + 1/n +2*Pt/s)
    term2num=Gmn(Theory,(1/m + 1/n)/2 -(P0+2*Ps)/s)
    term3num=Gmn(Theory,1/m + 1/n -2*Pt/s)
    term4num=Gmn(Theory,(1/m + 1/n)/2 -(P0-2*Ps)/s)
    Numerator = term0num*term1num*term2num*term3num*term4num

    term0den=2*Gtilde_mn(Theory,-P0/s)
    term1den=Gmn(Theory,1/m + 1/n + 2*Ps/s)
    term2den=Gmn(Theory,(1/m + 1/n)/2 - (P0+2*Pt)/s)
    term3den=Gmn(Theory,1/m + 1/n - 2*Ps/s)
    term4den=Gmn(Theory,(1/m + 1/n)/2 - (P0-2*Pt)/s)
    Denominator = term0den*term1den*term2den*term3den*term4den
    return Numerator/Denominator

#These are the plus/minus kernels/TV kernels
def KernelsM(Theory: Theory, P0: complex, Ps: complex, Pt: complex,
    epsilon: int):

    # epsilon =\pm are the elementary kernels while epsilon=0 is the
    # Teschner-Vartanov kernel
    s = Theory.s
    N = Theory.m * Theory.n # this is s^2 as an integer type

    zplus, zminus = rootsm(Theory, P0, Ps, Pt)

    prefactor = (
        M(Theory, P0, Ps, Pt)
        / (2j * smn(Theory, Pt) * smn(Theory, Ps) * mp.sqrt(
            quantum_determinantm(Theory, P0, Ps, Pt)))
    )

    bigsum = mp.mpc(0)

    if epsilon == 1:
        u0 = mp.log(zplus) / (2j * mp.pi * s**2)
        for k in range(N):

```

```

        bigsum += CurlyM(Theory, P0, Ps, Pt, u0 + mp.mpf(k) / N)
    return prefactor * bigsum

elif epsilon == -1:
    u0 = mp.log(zminus) / (2j * mp.pi * s**2)
    for k in range(N):
        bigsum += CurlyM(Theory, P0, Ps, Pt, u0 + mp.mpf(k) / N)
    return -prefactor * bigsum

elif epsilon == 0:
    return (KernelsM(Theory, P0, Ps, Pt, 1) + KernelsM(Theory, P0, Ps,
        Pt, -1)) / 2
else:
    raise ValueError("epsilon must be either 0, 1 or -1")

```

All the modules that we define in what follows correspond to identities of the modular kernels. At the moment, these are

- The identity limit (see (74)).
- The idempotency relation (see (??)).

```

#Consistency checks for the modular kernels

def Identitylimit(Theory: Theory, Ps: complex, Pt: complex):
    ident = -(Theory.b+1/Theory.b + 0.00001j)/2 # This is to avoid a
        fake pole
    return KernelsM(Theory, ident, Ps, Pt, 0), mp.sqrt(2)*mp.cos(4*mp.pi
        *Ps*Pt)

def Idempotencym(Theory: Theory, P0: complex, P1: complex, P2: complex,
    epsilon: int ):
    Integrand = lambda p: (
        KernelsM(Theory, P0, P1, 1j*(p+0.00001j), epsilon)*KernelsM(
            Theory, P0, 1j*(p+0.00001j), P2, epsilon)
    )

    LHS = mp.quad(Integrand, [-Theory.Lambda, Theory.Lambda])

    return LHS

```

Here is the battery of tests that we implemented.

```

# =====
# Demo tests
# =====

if DEMO:
    # --- Demo 1: shift identity youre currently testing ---
    z = mp.mpc(0.6+0.8j)
    z_plus = z + mp.mpf(1) / mp.mpf(theo.n)

    ratio = Gmn(theo, z_plus) / Gmn(theo, z)

    m_mp = mp.mpf(theo.m)
    rhs = mp.power(m_mp, mp.mpf(1)/2 - m_mp * z) \
        * mp.power(2 * mp.pi, (m_mp - mp.mpf(1)) / 2) \
        * mp.gamma(m_mp * z)

```

```

print("=== Demo: Gmn shift check ===")
print(f"ratio = {ratio}")
print(f"rhs = {rhs}")
print(f"relerr= {abs(ratio-rhs)/max(1,abs(rhs))}\n")

# --- Demo 2: conjugation & inversion checks for Gtilde_mn ---
Gv = Gtilde_mn(theo, z)
print("=== Demo: Gtilde_mn checks ===")
print(f"conj(Gtilde_mn(z)) = {mp.conj(Gv)}")
print(f"Gtilde_mn(conj(z)) = {Gtilde_mn(theo, mp.conj(z))}")
print(f"Gtilde_mn(-z) = {Gtilde_mn(theo, -z)}")
print(f"1/Gtilde_mn(z) = {1/Gv}")
print()

# --- Demo 3: alpha at zero point ---
alpha_demo = alphaf(theo, 0, 0, 0, 0, 0, 0)
print("=== Demo: alpha(0,0) ===")
print(f"alpha = {alpha_demo}")

#---momenta for consistency checks
P1=0.00001+0.01j
P2=0.00001+1j
P3=0.00001+0.03j
P4=0.00001+0.04j
Ps=0.00001+0.05j
Pt=0.00001+0.06j
P0=0.00001+0.07j
#--- Demo 4: Check that the roots obtained from the function roots(
Theory: Theory,Ps,Pt) are the same that the ones obtain from (-
beta \pm sqrt(beta^2-4*alpha*gamma)/2*alpha) ---
z_plus, z_minus = rootsf(theo, P1, P2, P3, P4, Ps, Pt)
alpha_val = alphaf(theo, P1, P2, P3, P4, Ps, Pt)
beta_val = betaf(theo, P1, P2, P3, P4, Ps, Pt)
gamma_val = gammaf(theo, P1, P2, P3, P4, Ps, Pt)
z_plus_check = (-beta_val + mp.sqrt(beta_val**2 - 4*alpha_val*
gamma_val)) / (2 * alpha_val)
z_minus_check = (-beta_val - mp.sqrt(beta_val**2 - 4*alpha_val*
gamma_val)) / (2 * alpha_val)
print("=== Demo: roots consistency check ===")
print(f"z_plus from roots() = {z_plus}")
print(f"z_plus from formula = {z_plus_check}")
print(f"z_minus from roots() = {z_minus}")
print(f"z_minus from formula = {z_minus_check}")

#--- Demo 5: Check the symmetry of CurlyF under the transformation
P_s\rightarrow P_3,P_3\rightarrow P_s,P_t\rightarrow P_1,P_1\
\rightarrow P_t ---
CurlyF_original = CurlyF(theo, P1, P2, P3, P4, Ps, Pt, z)
CurlyF_transformed = CurlyF(theo, Pt, P2, Ps, P4, P3, P1, z)
print("=== Demo: CurlyF symmetry check ===")
print(f"CurlyF original = {CurlyF_original}")
print(f"CurlyF transformed = {CurlyF_transformed}")

# --- Demo 6: Check the single-valuedness of the sum in the Kernels
function ---
z_plus, z_minus = rootsf(theo, P1, P2, P3, P4, Ps, Pt)

N = theo.m * theo.n

```

```

s = theo.s
ell = 3

res1 = mp.mpc(0)
res2 = mp.mpc(0)

u0 = mp.log(z_plus) / (2j * mp.pi * s**2)

for k in range(N):
    z1 = u0 + mp.mpf(k) / N
    z2 = u0 + mp.mpf(k + ell) / N
    res1 += CurlyF(theo, P1, P2, P3, P4, Ps, Pt, z1)
    res2 += CurlyF(theo, P1, P2, P3, P4, Ps, Pt, z2)

print("=== Demo: CurlyF single-valuedness check ===")
print(f"sum unshifted = {res1}")
print(f"sum shifted   = {res2}")

#--- Demo7: Check the reflection property of the spacelike kernels.
Fplus=Kernelsf(theo,P1,P2,P3,P4,Ps,Pt,1)
Fminus=Kernelsf(theo,P1,P2,P3,P4,Ps,Pt,-1)
Fplusreflected=Kernelsf(theo,P1,P2,P3,P4,-Ps,Pt,1)
Fminusreflected=Kernelsf(theo,P1,P2,P3,P4,-Ps,Pt,-1)
print("=== Demo: Reflection property of the plus/minus kernels ===")
print(f"Fplus = {Fplus}")
print(f"Fminus = {Fminus}")
print(f"Fplusreflected = {Fplusreflected}")
print(f"Fminusreflected = {Fminusreflected}")

#---Demo8: Check the reflection property of the Teschner-Vartanov
kernel
print("=== Demo: Reflection property of Teschner-Vartanov kernel
===")
print(f"FTeschnerVartanov = {Kernelsf(theo,P1,P2,P3,P4,Ps,Pt,0)}")
print(f"FTeschnerVartanovreflected = {Kernelsf(theo,P1,P2,P3,P4,-Ps,
Pt,0)}")

#---Demo9: Check the mixing symmetries of the different kernels
Fplusmixed=mixingsymmetrieskernelsf(theo,P1,P2,P3,P4,Ps,Pt,1)
Fminusmixed=mixingsymmetrieskernelsf(theo,P1,P2,P3,P4,Ps,Pt,-1)
print("=== Demo: mixing property of the kernels ===")
print(f"Fplus = {Fplus}")
print(f"Fplusmixed = {Fplusmixed}")
print(f"Fminus = {Fminus}")
print(f"Fminusmixed = {Fminusmixed}")

#---Demo10: Check Ioannis/Sylvain relation for the spacelike kernels
Fplusatonefourth=Kernelsf(theo,0.25,0.25,0.25,0.25,Ps,Pt,1)
Fminusatonefourth=Kernelsf(theo,0.25,0.25,0.25,0.25,Ps,Pt,-1)
Fplusfromrelation=RibaultTsiaresspacelikef(theo,Ps,Pt,1)
Fminusfromrelation=RibaultTsiaresspacelikef(theo,Ps,Pt,-1)
print("=== Demo: Ioannis/Sylvain relation for the spacelike kernels
at c=25 ===")
print(f"Fplusatonefourth = {Fplusatonefourth}")
print(f"Fplusfromrelation = {Fplusfromrelation}")
print(f"Fminusatonefourth = {Fminusatonefourth}")
print(f"Fminusfromrelation = {Fminusfromrelation}")

```

```

# ---Demo11: Check pentagon for the TV kernel
print("=== Demo: Testing the pentagon for the TeschnerVartanov
      kernel ===")
LHS, RHS = Pentagon(theo, parPenta, 0)
print(f"LHSPentagonplus = {LHS}")
print(f"RHSPentagonplus = {RHS}")

# ---Demo12: Check Idempotency for the TV kernel
print("=== Demo: Testing the Idempotency for the TeschnerVartanov
      kernel ===")
LHS= Idempotencyf(theo, parIdem, 0)
print(f"Idempotency = {LHS}")

# --- Demo 13: Check the single-valuedness of the sum in the modular
      Kernels function ---
z_plus, z_minus = rootsm(theo, P0, Ps, Pt)

N = theo.m * theo.n
s = theo.s
ell = 3

res1 = mp.mpc(0)
res2 = mp.mpc(0)

u0 = mp.log(z_plus) / (2j * mp.pi * s**2)

for k in range(N):
    z1 = u0 + mp.mpf(k) / N
    z2 = u0 + mp.mpf(k + ell) / N
    res1 += CurlyM(theo, P0, Ps, Pt, z1)
    res2 += CurlyM(theo, P0, Ps, Pt, z2)

print("=== Demo: CurlyM single-valuedness check ===")
print(f"sum unshifted = {res1}")
print(f"sum shifted   = {res2}")

# #--- Demo 14: Check the Identity limit of the Modular kernel
LHS, RHS = Identitylimit(theo, Ps, Pt)
print("=== Demo: Identity limit check ===")
print(f"Kernel at identity = {LHS}")
print(f"Value of the cos   = {RHS}")

# #--- Demo 15: Check reflection properties under reflection of the
      external momenta
print("=== Demo: Reflection property of Modular kernels ===")
print(f"Mplus = {KernelsM(theo,P0,Ps,Pt,1)}")
print(f"Mminus = {KernelsM(theo,P0,Ps,Pt,-1)}")
print(f"Mplusreflectedps = {KernelsM(theo,P0,-Ps,Pt,1)}")
print(f"Mminusreflectedps = {KernelsM(theo,P0,-Ps,Pt,-1)}")
print(f"Mplusreflectedpt = {KernelsM(theo,P0,Ps,-Pt,1)}")
print(f"Mminusreflectedpt = {KernelsM(theo,P0,Ps,-Pt,-1)}")
print(f"MTV = {KernelsM(theo,P0,Ps,Pt,0)}")
print(f"MTVreflectedps = {KernelsM(theo,P0,-Ps,Pt,0)}")
print(f"MTVreflectedpt = {KernelsM(theo,P0,Ps,-Pt,0)}")

```

Here I display the results of the tests.

```
=== Demo I: Consistency checks for the special functions ===
```

```

=== Demo: Gmn shift check ===
ratio = (0.49438518268587436096 - 0.48170611659036531661j)
rhs    = (0.49438518268587436096 - 0.48170611659036531661j)
=== Demo: Gtilde_mn checks ===
conj(Gtilde_mn(z))      = (0.48298133429787586343 -
    0.46023939521263494923j)
Gtilde_mn(conj(z))      = (0.48298133429787586343 -
    0.46023939521263494923j)
Gtilde_mn(-z)           = (1.0851287514495764127 - 1.0340337500226893283j
    )
1/Gtilde_mn(z)          = (1.0851287514495764127 - 1.0340337500226893283j
    )

=== Demo II: Consistency checks for the quantum modular fusion
    polynomial ===

=== Demo: alpha(0,0) ===
alpha = (0.0 + 4.1924144845297254444e-21j)
=== Demo: roots consistency check ===
z_plus from roots() = (-1.2611678633146055076 +
    0.000049540218341473836137j)
z_plus from formula = (-1.2611678633146055075 +
    0.000049540218341473822723j)
z_minus from roots() = (-0.96838174127724644965 +
    0.000062525639169306944851j)
z_minus from formula = (-0.96838174127724644976 +
    0.000062525639169306958265j)

=== Demo III: Consistency checks for the spacelike fusion kernels ===

=== Demo: CurlyF symmetry check ===
CurlyF original      = (1.0004160450050665359 + 0.02442251129663564322j)
CurlyF transformed   = (1.0004160450050665359 + 0.02442251129663564322j)
=== Demo: CurlyF single-valuedness check ===
sum unshifted = (0.9999938049400514083 - 0.0028236456207856117867j)
sum shifted   = (0.99999380494005140954 - 0.0028236456207856147357j)
=== Demo: Reflection property of the plus/minus kernels ===
Fplus = (0.0070379226124723145446 + 6.7370768266815472853j)
Fminus = (0.0081042804008412503518 - 6.7370818422901804711j)
Fplusreflected = (0.0081042804008412502499 - 6.7370818422901804711j)
Fminusreflected = (0.0070379226124723090828 + 6.7370768266815472775j)
=== Demo: Reflection property of Teschner-Vartanov kernel ===
FTeschnerVartanov = (0.0075711015066567824482 - 2.5078043165928696296e-6
    j)
FTeschnerVartanovreflected = (0.0075711015066567796664 -
    2.5078043165968235794e-6j)
=== Demo: mixing property of the kernels ===
Fplus = (0.0070379226124723145446 + 6.7370768266815472853j)
Fplusmixed = (0.0081054953745872160188 - 6.7370818411835346656j)
Fminus = (0.0081042804008412503518 - 6.7370818422901804711j)
Fminusmixed = (0.0070367076404285165413 + 6.737076828305664042j)
=== Demo: Ioannis/Sylvain relation for the spacelike kernels at c=25 ===
Fplusatonefourth = (0.022508560047440773346 + 1.1961257192910219677j)
Fplusfromrelation = (0.022508560047440773356 + 1.1961257192910219677j)
Fminusatonefourth = (0.02258994051224826017 - 1.1961407251585321349j)
Fminusfromrelation = (0.02258994051224826016 - 1.1961407251585321348j)
=== Demo: Testing the pentagon for the TeschnerVartanov kernel ===
LHSPentagonplus = (0.040478458528855747172 + 0.000094724396361210183938j

```

```

)
RHSPentagonplus = (0.040480286896388033099 + 0.000094758445189765142584j
)
=== Demo: Testing the Idempotency for the TeschnerVartanov kernel ===
Idempotency = (-1.3517102240608496172e-8 - 9.1919226111705190442e-8j)

=== Demo IV: Consistency checks for the spacelike modular kernels ===

=== Demo: CurlyM single-valuedness check ===
sum unshifted = (1.3609325834244597261 - 0.14924114543484447604j)
sum shifted   = (1.3609325834244597259 - 0.14924114543484447632j)
=== Demo: Identity limit check ===
Kernel at identity = (1.4132087274245876136 - 6.9794754220596457469e-6j)
Value of the cos   = (1.4132087251469752577 + 7.3679348136474341459e-7j)
=== Demo: Reflection property of Modular kernels ===
Mplus = (0.063138621188207340149 - 0.82235300568100736394j)
Mminus = (0.063378389776097701905 + 0.8223112349827522751j)
Mplusreflectedps = (0.063378389776097701907 + 0.8223112349827522751j)
Mminusreflectedps = (0.0631386211882073401 - 0.82235300568100736392j)
Mplusreflectedpt = (0.063378389776097701906 + 0.8223112349827522751j)
Mminusreflectedpt = (0.063138621188207340149 - 0.82235300568100736394j)
MTV = (0.063258505482152521027 - 0.000020885349127544417484j)
MTVreflectedps = (0.063258505482152521003 - 0.00002088534912754440605j)
MTVreflectedpt = (0.063258505482152521027 - 0.000020885349127544417484j)

=== Demo V: important identities involving the spacelike modular and
fusion kernels ===

=== Demo: Non rational Verlinde formula ===
LHSVerlindeformula = (0.0018437816707970328695 - 1.2143930559352561683e
-6j)
RHSVerlindeformula = (0.0018361623980578999904 - 1.2095319899761086453e
-6j)

```